

C++ OBJECT ORIENTED PROGRAMMING

Comprehensive Study Notes — Unit I & Unit II

UNIT - I | Introduction to OOP | 10 Hours

1. Introduction to Object Oriented Programming

1.1 Need for OOP

Traditional procedural programming organizes code around functions. As programs grow larger, managing global data, code reuse, and complexity becomes very difficult. OOP was introduced to model real-world problems naturally.

Problems with Procedural Programming:

- Global data accessible by any function — prone to unintended modification
- Functions and data are separate — no natural grouping for real-world entities
- Code reuse is hard — copy-pasting leads to inconsistency and bugs
- No access restriction — any function can accidentally corrupt any data
- Difficult to scale and maintain large codebases

How OOP Solves These:

- Bundles data + functions into objects — mirrors real-world entities
- Data is protected via access specifiers (private/protected/public)
- Inheritance enables code reuse without duplication
- Polymorphism allows one interface to handle many different data types
- Modular design — each class is independent and testable

1.2 Comparison of Programming Paradigms

Feature	Procedural	Object Oriented	Functional
Focus	Functions/Procedures	Objects & Classes	Pure Functions
Data	Shared global	Encapsulated in objects	Immutable
Reuse	Function calls	Inheritance	Higher-order functions
Examples	C, Pascal	C++, Java, Python	Haskell, Erlang
Key Feature	Top-down design	Encapsulation	No side effects

1.3 Characteristics of OOP Languages

1. Class — Blueprint for objects

A class defines the structure (attributes) and behavior (methods) for objects. It does not occupy memory by itself.

2. Object — Instance of a class

Each object has its own copy of data members. Objects interact via member function calls (message passing).

3. Encapsulation

Wrapping data and functions into one unit (class). Data is hidden from direct outside access. Achieved via access specifiers.

4. Data Abstraction

Showing only essential features, hiding implementation. Users see 'what' — not 'how'. Achieved via abstract classes/interfaces.

5. Inheritance

A derived class acquires properties of a base class. Promotes 'is-a' relationships and code reuse.

6. Polymorphism

'Many forms' — same function/operator behaves differently based on context. Two types: Compile-time (overloading) and Run-time (virtual functions).

7. Dynamic Binding

Function to call is determined at runtime. Enabled by virtual functions and base class pointers.

8. Message Passing

Objects communicate by calling each other's functions. Message = object + function name + parameters.

1.4 Structure of a C++ Program

```
// ===== STRUCTURE OF A C++ PROGRAM =====

#include <iostream>          // 1. Preprocessor directive (includes header)
#include <string>

using namespace std;       // 2. Namespace (avoids std:: prefix)

int globalVar = 10;        // 3. Global declaration

void greet(string name);   // 4. Function prototype

class MyClass {            // 5. Class definition
private:
    int data;
public:
    MyClass(int d) : data(d) {}
```

```

    void display() { cout << "Data: " << data << endl; }
};

int main() {
    // 6. main() – entry point
    MyClass obj(42);
    obj.display();           // Data: 42
    greet("Alice");
    return 0;                // 7. Return status to OS
}

void greet(string name) { // 8. Function definition
    cout << "Hello, " << name << "!" << endl;
}

```

1.5 Use of cin and cout

```

#include <iostream>
using namespace std;

int main() {
    int age; string name;

    // cout: standard output stream – uses << (insertion operator)
    cout << "Enter your name: ";

    // cin: standard input stream – uses >> (extraction operator)
    cin >> name;

    cout << "Enter age: ";
    cin >> age;

    // Chain multiple outputs
    cout << "Hello " << name << ", Age: " << age << endl;

    // Read full line (including spaces)
    cin.ignore();           // Clear remaining newline
    string sentence;
    cout << "Enter sentence: ";
    getline(cin, sentence); // Read entire line
    cout << "You said: " << sentence << endl;

    // Formatting output
    cout << fixed;           // Fixed decimal notation
    cout.precision(2);
    cout << 3.14159;        // Outputs: 3.14
    return 0;
}

```

1.6 Compilation Process in C++

C++ compilation converts source code to an executable through 4 stages:

Step	Stage	Tool	Output
1	Preprocessing	cpp (preprocessor)	Expanded source (.i) — macros & #include resolved
2	Compilation	g++ compiler	Assembly code (.s) — syntax checked
3	Assembly	as (assembler)	Object code (.o) — machine code, unlinked
4	Linking	ld (linker)	Executable — libraries attached, final binary

```
# Compile commands:
g++ program.cpp -o program      # Full compilation in one command
g++ -E program.cpp              # Preprocessing only
g++ -S program.cpp              # Compile to assembly (.s file)
g++ -c program.cpp              # Compile to object file (.o)
g++ program.o -o program        # Link object file to executable
```

2. C++ Programming Language (Procedural)

2.1 Tokens

Tokens are the smallest meaningful units recognized by the C++ compiler.

Token Type	Description	Examples
Keywords	Reserved words with special meaning	int, float, class, if, while, return, new
Identifiers	User-defined names	myVar, calculate, Student, _count
Literals	Constant values	42, 3.14, 'A', "hello", true, nullptr
Operators	Symbols for operations	+ - * / % == != && << >>
Punctuators	Structural symbols	{ } () [] ; , . :: ->

2.2 Data Types in C++

Basic (Fundamental) Data Types

```
// === INTEGER TYPES ===
short int    s = 32767;           // 2 bytes | Range: -32768 to 32767
int          i = 2147483647;      // 4 bytes | Range: -2^31 to 2^31-1
long int     l = 2147483647L;     // 4 or 8 bytes
long long    ll = 9223372036854775807LL; // 8 bytes
unsigned int ui = 4294967295U;    // 4 bytes | Range: 0 to 2^32-1

// === FLOATING POINT TYPES ===
float        f = 3.14f;           // 4 bytes | ~7 decimal digits
double       d = 3.141592653589;  // 8 bytes | ~15 decimal digits
long double  ld = 3.14159265358979L; // 10-16 bytes | highest precision
```

```
// === CHARACTER TYPE ===
char c = 'A'; // 1 byte | stores ASCII value (65 for 'A')
wchar_t wc = L'A'; // 2-4 bytes | wide character for unicode

// === BOOLEAN TYPE ===
bool b = true; // 1 byte | true(1) or false(0)

// === VOID TYPE ===
// void – no type. Used for functions that return nothing.
```

Advanced (User-Defined) Data Types

```
// STRUCTURE – groups different types together
struct Student { int rollNo; string name; float marks; };
Student s = {101, "Alice", 95.5f};

// UNION – all members share same memory location
union Data { int i; float f; char c; };
Data d; d.i = 65;
cout << d.c; // 'A' – same memory, different interpretation

// ENUMERATION – named integer constants
enum Color { RED, GREEN, BLUE }; // RED=0, GREEN=1, BLUE=2
enum Day { MON=1, TUE, WED, THU }; // Custom start value
Color c = GREEN;

// CLASS (user-defined type – covered in Unit II)
class Rectangle { int width, height; /* ... */ };
```

Derived Data Types

```
// ARRAYS – collection of same-type elements
int arr[5] = {10, 20, 30, 40, 50}; // 1D array
int mat[2][3] = {{1,2,3},{4,5,6}}; // 2D array
cout << arr[2]; // 30 (zero-indexed)

// POINTERS – stores memory address
int val = 100;
int* ptr = &val; // ptr stores address of val
cout << *ptr; // Dereference: outputs 100
cout << ptr; // Outputs memory address

// REFERENCES (alias – same memory, different name)
int& ref = val; // ref IS val
ref = 200;
cout << val; // 200 – val was modified via ref!

// FUNCTIONS are also derived types
int add(int a, int b) { return a + b; }
```

2.3 Variables

```
int age;           // Declared but uninitialized (garbage value!)
int score = 0;     // Declared and initialized
int a=1, b=2, c=3; // Multiple in one line

const double PI = 3.14159265; // Constant – cannot be changed

// SCOPE:
int globalVar = 10; // Global – accessible everywhere

void demo() {
    int localVar = 20; // Local – only in this function
    {
        int blockVar = 30; // Block – only in this { }
    }
    // blockVar NOT accessible here
}

// STATIC variable – retains value across function calls
void counter() {
    static int count = 0; // Initialized ONCE
    count++;
    cout << count;       // Outputs 1, 2, 3... on successive calls
}

// REGISTER variable (hint to compiler)
register int fast = 0; // Suggest storing in CPU register (modern compilers
                      // ignore this)

// EXTERN – declared in another file
extern int sharedVar; // Definition is in another .cpp file
```

2.4 Reference vs Pointers

Feature	Reference	Pointer
Syntax	int& ref = var;	int* ptr = &var;
Null value	Cannot be null	Can be nullptr
Reassignment	Cannot reassign (fixed alias)	Can point to another var
Memory address	No own address (alias)	Has its own address
Dereferencing	Automatic (use directly)	Manual (* needed)
Arithmetic	Not allowed	Pointer arithmetic allowed
Initialization	Must init when declared	Can declare without init

```
#include <iostream>
```

```

using namespace std;

void swapByRef(int& a, int& b) { // Swap using references
    int temp = a; a = b; b = temp;
}

void swapByPtr(int* a, int* b) { // Swap using pointers
    int temp = *a; *a = *b; *b = temp;
}

int main() {
    int x = 10, y = 20;

    // Reference
    int& rx = x;
    rx = 99;
    cout << x;          // 99 - x was modified via rx

    // Pointer
    int* px = &x;
    *px = 50;
    cout << x;          // 50
    px = &y;            // Pointer can be reassigned
    cout << *px;        // 20

    // Pointer arithmetic
    int arr[] = {10, 20, 30, 40};
    int* p = arr;
    cout << *(p+2);    // 30 (moves 2 ints forward)
    p++;
    cout << *p;        // 20

    // Swap demos
    swapByRef(x, y);   cout << x << " " << y;
    swapByPtr(&x, &y); cout << x << " " << y;
}

```

2.5 Operators

Scope Resolution Operator (::)

```

int x = 100;    // Global variable

class Counter {
public:
    int x;
    void setValue(int v); // Declared inside
    void show();
};

// Defined OUTSIDE class - :: specifies which class

```

```

void Counter::setValue(int v) { x = v; }

void Counter::show() {
    cout << "Member x: " << x << endl;
    cout << "Global x: " << ::x << endl; // :: accesses global
}

int main() {
    Counter c;
    c.setValue(50);
    c.show();
    // Member x: 50
    // Global x: 100
}

```

Dynamic Memory Operators (new / delete)

```

int main() {
    // new: allocates on HEAP (manual lifetime control)
    int* p = new int;           // Allocate single int
    *p = 42;
    cout << *p;                 // 42

    int* q = new int(99);       // Allocate and initialize
    cout << *q;                 // 99

    int n = 5;
    int* arr = new int[n];      // Dynamic array
    for(int i = 0; i < n; i++) arr[i] = i * 10;

    // delete: frees heap memory (prevent memory leaks!)
    delete p;                   // Free single variable
    delete q;
    delete[] arr;              // MUST use [] for arrays

    p = nullptr;               // Best practice: nullify after delete
}

// WARNING: Not calling delete = memory leak!
// double-delete or delete[] on non-array = undefined behavior

```

Memory: Stack vs Heap

Stack: Local variables, automatic management, limited size, LIFO order. Heap: Dynamic allocation with new/delete, manually managed, larger size, can persist beyond function scope.

Typecast Operator

```

// C-style cast (old, unsafe)
double d = 9.7;
int i = (int)d;                // i = 9 (truncated)

```



```

// C++ style casts (preferred):

// static_cast – compile-time, safe conversions
double pi = 3.14159;
int n = static_cast<int>(pi);           // 3
double res = static_cast<double>(7)/2; // 3.5 (not integer div)

// dynamic_cast – safe downcasting in inheritance (needs virtual)
// Base* b = new Derived();
// Derived* d = dynamic_cast<Derived*>(b); // null if fails

// const_cast – add/remove const qualifier
const int val = 10;
int* modifiable = const_cast<int*>(&val);

// reinterpret_cast – raw bit reinterpretation (dangerous!)
int num = 65;
char* ch = reinterpret_cast<char*>(&num); // treats int as char

```

2.6 Expressions

```

int a=5, b=3;

// ARITHMETIC
int sum = a + b;      // 8
int diff= a - b;      // 2
int prod= a * b;      // 15
int quot= a / b;      // 1 (integer division – truncates!)
int rem = a % b;      // 2 (remainder)
double fdiv = (double)a/b; // 1.6667

// RELATIONAL (return bool)
bool eq = (a==b);    // false
bool gt = (a>b);     // true
bool ne = (a!=b);    // true

// LOGICAL
bool t=true, f=false;
bool r1 = t && f;    // false (AND)
bool r2 = t || f;   // true  (OR)
bool r3 = !t;       // false (NOT)

// BITWISE
int x=5; // 0101
int y=3; // 0011
cout << (x&y);      // 0001 = 1 (AND)
cout << (x|y);      // 0111 = 7 (OR)
cout << (x^y);      // 0110 = 6 (XOR)

```

```

cout << (x<<1); // 1010 = 10 (left shift, *2)
cout << (x>>1); // 0010 = 2 (right shift, /2)

// TERNARY
int max = (a>b) ? a : b; // a if a>b, else b

// ASSIGNMENT COMPOUND
int n=10;
n += 5; // n = 15
n -= 3; // n = 12
n *= 2; // n = 24
n /= 4; // n = 6
n %= 4; // n = 2

// INCREMENT/DECREMENT
int c=5;
cout << c++; // 5 (post: returns OLD value, then increments)
cout << ++c; // 7 (pre: increments first, then returns)
cout << c--; // 7 (post: returns OLD value, then decrements)
cout << --c; // 5 (pre: decrements first, then returns)

```

2.7 Functions

Basic Function Structure

```

returnType functionName(param1Type param1, param2Type param2) {
    // body
    return value; // omit if void
}

// Example:
int multiply(int a, int b) { return a * b; }

// Void function:
void printLine() { cout << "-----" << endl; }

// Returning reference:
int& getElement(int arr[], int i) { return arr[i]; }

```

Inline Functions

An inline function is a request to the compiler to insert the function body at each call site, eliminating function call overhead. Best for small, frequently-called functions.

```

inline int square(int x) { return x * x; }
inline double area(double r) { return 3.14159 * r * r; }

int main() {
    // Compiler may REPLACE: int s = square(5);
    // with:                  int s = 5 * 5;
    cout << square(5); // 25
}

```

```

    cout << area(7.0);    // 153.938
}

// Member functions defined INSIDE a class are automatically inline
class Demo {
public:
    int getVal() { return val; } // Auto-inline
private: int val;
};

// inline is a HINT – compiler may ignore for:
// • recursive functions
// • functions with loops
// • very large functions

```

Const Arguments

const parameters prevent modification inside the function. Especially useful with references to avoid copying while protecting data.

```

// const by VALUE – copy is const, but original unchanged anyway
void printAge(const int age) {
    // age = 25; // ERROR: cannot modify const parameter
    cout << age;
}

// const REFERENCE – efficient (no copy) AND safe (no modification)
void displayInfo(const string& name, const double& gpa) {
    // name = "Bob"; // ERROR
    cout << name << ": " << gpa << endl;
}

// Particularly useful for large objects:
void processData(const vector<int>& data) { // No copy of entire vector
    for(const int& val : data)
        cout << val << " ";
}

// const return type
const double getPI() { return 3.14159265358979; }
// const double pi = getPI();
// pi = 3.14; // ERROR

```

Default Arguments

Default values allow calling a function without providing all arguments. Defaults must be in the rightmost parameters.

```

// Defaults specified in DECLARATION
void greet(string name, string msg = "Hello", char end = '!');

void greet(string name, string msg, char end) { // No defaults in definition

```

```

    cout << msg << ", " << name << end << endl;
}

// Practical example
void drawBox(int width=10, int height=5, char sym='*') {
    for(int i=0;i<height;i++) {
        for(int j=0;j<width;j++) cout << sym;
        cout << endl;
    }
}

int main() {
    greet("Alice");           // Hello, Alice!
    greet("Bob", "Hi");       // Hi, Bob!
    greet("Carol", "Good day", '.'); // Good day, Carol.

    drawBox();                // 10 wide, 5 tall, stars
    drawBox(5);               // 5 wide, 5 tall, stars
    drawBox(8, 3, '#');       // 8 wide, 3 tall, hashes
}

// RULE: Once a default is given, ALL params to the RIGHT must have defaults
// void wrong(int a=5, int b); // ERROR: b must also have default
// void ok(int a, int b=5);    // OK

```

UNIT - II | Classes and Objects | Polymorphism | 11 Hours

3. Classes and Objects

3.1 Objects and Classes — Detailed

```

#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    int rollNo;
    string name;
    float marks;

public:
    // Constructor
    Student(int r, string n, float m) : rollNo(r), name(n), marks(m) {}

    // Getter functions

```

```

int getRoll() const { return rollNo; }
string getName() const { return name; }
float getMarks() const { return marks; }

// Setter functions
void setMarks(float m) { if(m>=0&&m<=100) marks=m; }

void display() const {
    cout << rollNo << " | " << name << " | " << marks << endl;
}

bool operator>(const Student& s) const { return marks > s.marks; }
};

int main() {
    // Stack objects
    Student s1(101, "Alice", 95.5f);
    Student s2(102, "Bob", 88.0f);

    s1.display(); // 101 | Alice | 95.5
    s2.display(); // 102 | Bob | 88

    // Heap object
    Student* s3 = new Student(103, "Carol", 92.0f);
    s3->display(); // Arrow operator for pointer
    delete s3;

    // Array of objects
    Student arr[2] = { Student(104, "Dave", 75.0f), Student(105, "Eve", 88.0f) };
    for(int i=0; i<2; i++) arr[i].display();
}

```

3.2 Encapsulation

Encapsulation bundles data and methods, restricting direct external access to internal data. Implemented via private data + public interface.

```

class BankAccount {
private:
    string accountNo;
    double balance;    // HIDDEN from outside
    string pin;

public:
    BankAccount(string acc, double init, string p)
        : accountNo(acc), balance(init), pin(p) {}

    // CONTROLLED access
    void deposit(double amount) {
        if(amount > 0) balance += amount;
        else cout << "Invalid amount!" << endl;
    }
}

```

```

    }

    bool withdraw(double amount, string enteredPin) {
        if(enteredPin != pin) { cout << "Wrong PIN!\n"; return false; }
        if(amount > balance) { cout << "Insufficient funds!\n"; return false; }
        balance -= amount;
        return true;
    }

    double getBalance(string p) const {
        if(p == pin) return balance;
        cout << "Invalid PIN!" << endl; return -1;
    }
};

int main() {
    BankAccount acc("ACC001", 1000.0, "1234");
    // acc.balance = 99999; // ERROR: private!
    acc.deposit(500);
    cout << acc.getBalance("1234"); // 1500
}

```

3.3 Data Abstraction

Abstraction = show only what the user needs, hide the implementation. Users interact with a simple interface; internal complexity is invisible.

```

class Stack {
private:
    int arr[100];    // Implementation hidden
    int top = -1;

public:
    void push(int v) {
        if(top < 99) arr[++top] = v;
        else cout << "Stack overflow!\n";
    }
    int pop() {
        if(top >= 0) return arr[top--];
        cout << "Stack underflow!\n"; return -1;
    }
    int peek() const { return top>=0 ? arr[top] : -1; }
    bool isEmpty() const { return top == -1; }
    int size() const { return top + 1; }
};

int main() {
    Stack s;
    s.push(10); s.push(20); s.push(30);
    cout << s.peek(); // 30
    cout << s.pop(); // 30 – HOW it stores is abstracted away
}

```

```
    cout << s.size(); // 2
}
```

3.4 Private, Public, and Protected Access Specifiers

Specifier	Accessible From	Primary Use
private	Only within the SAME class	Data members — enforce encapsulation
public	Everywhere (same class, derived, outside)	Interface methods, constructors
protected	Same class AND derived classes (not outside)	Inheritance — controlled shared access

3.5 Memory Organization of a Class

Understanding memory layout helps write efficient code:

- Non-static data members: Each object gets its own copy (on stack or heap)
- Member functions: Stored ONCE in code segment, shared by all objects
- Static data members: Stored in data segment, shared by ALL objects of that class
- 'this' pointer: Implicit pointer to the current object, available in all non-static member functions

```
class Demo {
public:
    int x;           // Per-object (non-static)
    static int count; // Class-wide (static) — stored ONCE
    void show() { cout << x; } // Code stored once, shared
};
int Demo::count = 0; // Static member definition outside class

int main() {
    Demo d1, d2;
    d1.x = 10; d2.x = 20;
    cout << d1.x; // 10
    cout << d2.x; // 20 — own copy
    Demo::count = 5;
    cout << d1.count << d2.count; // 5 5 — shared!
    cout << sizeof(Demo); // Only size of non-static data (int = 4 bytes)
}

// this pointer:
class MyClass {
    int val;
public:
    void set(int val) { this->val = val; } // this->val is member, val is param
    MyClass& chain(int v) { val=v; return *this; } // Return self for chaining
};
```

3.6 Member Functions — Inline vs Non-Inline

```
class Circle {
private:
    double radius;

public:
    // === INLINE: defined INSIDE class body (auto-inline) ===
    Circle(double r) : radius(r) {}
    double getRadius() const { return radius; } // Auto-inline

    // === NON-INLINE: declared inside, defined OUTSIDE ===
    double area();
    double circumference();
    void display();
    inline void scale(double f); // Explicitly inline, defined outside
};

// Non-inline definitions
double Circle::area() { return 3.14159 * radius * radius; }
double Circle::circumference() { return 2 * 3.14159 * radius; }

void Circle::display() {
    cout << "r=" << radius << " area=" << area()
         << " circ=" << circumference() << endl;
}

inline void Circle::scale(double f) { radius *= f; }

int main() {
    Circle c(5.0);
    c.display(); // r=5 area=78.5398 circ=31.4159
    c.scale(2.0);
    c.display(); // r=10 area=314.159 circ=62.8318
}
```

3.7 Static Member Variables and Functions

```
class Counter {
private:
    int id;
    static int count; // Shared across ALL objects

public:
    Counter() { count++; id = count; }
    ~Counter() { count--; }
    int getId() const { return id; }

    // Static function — can ONLY access static members
    static int getCount() {
        return count;
    }
}
```



```

        // return id; // ERROR: id is non-static, no 'this' in static
    }
};

int Counter::count = 0;    // Must define outside class

int main() {
    cout << Counter::getCount(); // 0 – no object needed for static
    Counter c1;
    Counter c2;
    cout << Counter::getCount(); // 2
    {
        Counter c3;
        cout << Counter::getCount(); // 3
    } // c3 destroyed
    cout << Counter::getCount(); // 2
}

```

3.8 Friend Functions

A friend function is NOT a member but can access private/protected members. Declared with 'friend' keyword inside the class.

```

class Box {
private:
    double l, w, h;

public:
    Box(double l, double w, double h) : l(l), w(w), h(h) {}

    friend double volume(const Box& b);           // Friend function
    friend class BoxPrinter;                     // Friend class

    friend bool operator==(const Box& a, const Box& b); // Friend operator
};

double volume(const Box& b) {
    return b.l * b.w * b.h; // Can access private members!
}

class BoxPrinter {
public:
    void print(const Box& b) {
        cout << b.l << "x" << b.w << "x" << b.h << endl; // Access private
    }
};

bool operator==(const Box& a, const Box& b) {
    return a.l==b.l && a.w==b.w && a.h==b.h;
}

```

```
int main() {
    Box b1(3,4,5), b2(3,4,5);
    cout << volume(b1);    // 60
    BoxPrinter bp; bp.print(b1); // 3x4x5
    cout << (b1 == b2);    // 1 (true)
}
```

Friend Function Rules

- Friendship is NOT inherited — Derived class of a friend is NOT a friend
- Friendship is NOT transitive — friend of friend is NOT a friend
- Use sparingly — it breaks encapsulation; mainly needed for operator overloading

3.9 Class vs Structure

```
struct Point {
    int x, y;           // public by DEFAULT
    void print() { cout << x << ", " << y; } // public by default
};

class Point2D {
    int x, y;           // private by DEFAULT
public:
    Point2D(int a, int b) : x(a), y(b) {}
    void print() { cout << x << ", " << y; }
};

// Both are otherwise equivalent in C++
// struct is typically used for plain-old-data (POD)
// class is typically used for full OOP design
```

3.10 Constructors — All Types

Default Constructor

```
class Rectangle {
    int w, h;
public:
    Rectangle() : w(0), h(0) { // Default — no parameters
        cout << "Default constructor" << endl;
    }
    int area() { return w * h; }
};

// If NO constructor defined, compiler generates an empty default
// If ANY constructor is defined, compiler's default is REMOVED

Rectangle r; // Default constructor called
```

Parameterized Constructor

```
class Rectangle {
    int w, h;
public:
    Rectangle(int width, int height) : w(width), h(height) {}
    int area() { return w * h; }
    int perimeter() { return 2*(w+h); }
};

Rectangle r1(10, 5);           // Direct initialization
Rectangle r2 = Rectangle(3,7); // Copy initialization
Rectangle* r3 = new Rectangle(4,6); // Dynamic
cout << r1.area();    // 50
delete r3;
```

Copy Constructor

```
class Student {
    string name;
    int* grades;
    int n;
public:
    Student(string nm, int num) : name(nm), n(num) {
        grades = new int[n];
        for(int i=0;i<n;i++) grades[i]=0;
    }

    // Copy constructor – MUST do deep copy for dynamic memory!
    Student(const Student& other) : name(other.name), n(other.n) {
        grades = new int[n];           // New allocation!
        for(int i=0;i<n;i++) grades[i]=other.grades[i];
        cout << "Copy constructor called" << endl;
    }

    void setGrade(int i, int g) { grades[i]=g; }
    void display() { cout<<name<<": "; for(int i=0;i<n;i++) cout<<grades[i]<<" ";
    cout<<endl; }
    ~Student() { delete[] grades; }
};

int main() {
    Student s1("Alice", 3);
    s1.setGrade(0,90); s1.setGrade(1,85); s1.setGrade(2,92);

    Student s2 = s1;           // Copy constructor – DEEP copy
    s2.setGrade(0,100);       // Modifying s2 does NOT affect s1

    s1.display(); // Alice: 90 85 92
    s2.display(); // Alice: 100 85 92
}
```

```
// SHALLOW copy (default) – copies pointer value
// Both objects point to SAME memory – double-delete crash!
// DEEP copy (custom) – allocates new memory, copies data
// Objects are completely independent – safe!
```

Dynamic Constructor

```
class Matrix {
    int** data;
    int rows, cols;
public:
    // Dynamic constructor – allocates 2D array on heap
    Matrix(int r, int c) : rows(r), cols(c) {
        data = new int*[r];
        for(int i=0;i<r;i++) {
            data[i] = new int[c];
            for(int j=0;j<c;j++) data[i][j] = 0;
        }
        cout << "Matrix " << r << "x" << c << " created" << endl;
    }
    void set(int r, int c, int v) { data[r][c]=v; }
    int get(int r, int c) const { return data[r][c]; }

    ~Matrix() {
        for(int i=0;i<rows;i++) delete[] data[i];
        delete[] data;
        cout << "Matrix destroyed" << endl;
    }
};

int main() {
    Matrix m(3,3);
    m.set(0,0,1); m.set(1,1,5); m.set(2,2,9);
    cout << m.get(1,1); // 5
}
```

3.11 Destructors

```
class FileHandler {
    string filename;
    bool isOpen;
public:
    FileHandler(string f) : filename(f), isOpen(true) {
        cout << "Opened: " << filename << endl;
    }

    // Destructor: ~ClassName() – no params, no return type
    ~FileHandler() {
        if(isOpen) {
            isOpen = false;
        }
    }
};
```

```

        cout << "Closed: " << filename << endl;
    }
}

};

int main() {
    FileHandler f1("data.txt");    // Opened: data.txt
    {
        FileHandler f2("log.txt"); // Opened: log.txt
        FileHandler f3("err.txt"); // Opened: err.txt
    } // f3 closed, f2 closed ← LIFO (Last In First Out) order
    cout << "Back in outer scope" << endl;
} // f1 closed

// Order of destruction is REVERSE of construction (stack unwinding)

```

3.12 Assignment Operator Overloading

```

class Vector {
    double* data;
    int size;
public:
    Vector(int n) : size(n) { data = new double[n](); }

    // Assignment operator
    Vector& operator=(const Vector& other) {
        if(this == &other) return *this; // Self-assignment guard!

        delete[] data;                // Free existing memory
        size = other.size;
        data = new double[size];      // New allocation
        for(int i=0;i<size;i++) data[i] = other.data[i];

        cout << "Assignment operator called" << endl;
        return *this;                 // Enable chaining: a = b = c
    }

    void set(int i, double v) { data[i]=v; }
    double get(int i) const { return data[i]; }
    ~Vector() { delete[] data; }
};

int main() {
    Vector v1(3); v1.set(0,1); v1.set(1,2); v1.set(2,3);
    Vector v2(3);
    v2 = v1;    // Assignment operator called

    Vector v3(3), v4(3);
    v4 = v3 = v1; // Chained: v3=v1 first, then v4=v3
}

```

4. Polymorphism in Detail

Polymorphism (Greek: 'many forms') allows the same interface to work with different data types or behave differently based on context.

4.1 Function Overloading

```
// Same name, DIFFERENT parameter lists
int    add(int a, int b)        { return a+b; }
double add(double a, double b) { return a+b; }
int    add(int a, int b, int c){ return a+b+c; }
string add(string a, string b) { return a+b; }

// Compiler selects based on argument types at COMPILE TIME
cout << add(3, 4);              // int version → 7
cout << add(3.5, 4.2);          // double version → 7.7
cout << add(1, 2, 3);           // 3-param version → 6
cout << add("Hi", " World");    // string version → Hi World

// CANNOT overload on return type alone:
// int foo() {}    // ERROR: both have same params
// double foo() {}
```

4.2 Constructor Overloading

```
class Date {
    int day, month, year;
public:
    Date() : day(1), month(1), year(2000) {}           // Default
    Date(int y) : day(1), month(1), year(y) {}         // Year only
    Date(int d, int m, int y) : day(d), month(m), year(y) {} // Full
    Date(const Date& d) : day(d.day), month(d.month), year(d.year) {} // Copy

    void display() { cout<<day<<"/"<<month<<"/"<<year<<endl; }
};

Date d1;          // 1/1/2000
Date d2(2025);    // 1/1/2025
Date d3(25,12,2024); // 25/12/2024
Date d4 = d3;     // Copy
```

4.3 Compile-Time Polymorphism

Compile-Time vs Run-Time Polymorphism

COMPILE-TIME (Static/Early binding): • Resolved by compiler based on parameter types • Fast — no runtime overhead • Achieved via: Function overloading, Operator overloading
RUN-TIME (Dynamic/Late binding): • Resolved during execution via vtable lookup • Slight overhead • Achieved via: virtual functions, function overriding

4.4 Overloading Rules

- Functions MUST differ in parameter list (number, type, or order)
- Return type ALONE does NOT distinguish overloaded functions
- Default arguments can cause ambiguity — use carefully
- const and non-const member functions can be overloaded
- All overloaded functions must be in the same scope

```
// VALID overloads:
void f(int a);
void f(double a);           // Different type
void f(int a, int b);       // Different count
void f(int a, double b);    // Different types
void f(double a, int b);    // Different order

// AMBIGUITY example — avoid this pattern:
void g(int a, int b=0);
void g(int a);
// g(5); → ERROR: ambiguous — which g?
```

4.5 Operator Overloading — All Key Operators

Operators That CANNOT Be Overloaded

:: (scope resolution) . (member access) .* (member pointer) ?: (ternary) sizeof typeid

Member vs Friend: When to Use Which

Use MEMBER function when: left operand is object of your class (a+b, ++a, a[i], a())
Use FRIEND function when: left operand is NOT your class (cin>>obj, cout<<obj, 5+Complex)

Arithmetic Operators (+, -, *, /)

```
class Complex {
    double real, imag;
public:
    Complex(double r=0, double i=0): real(r), imag(i) {}

    // Binary + as member
    Complex operator+(const Complex& c) const {
        return Complex(real+c.real, imag+c.imag);
    }
    Complex operator-(const Complex& c) const {
```

```

        return Complex(real-c.real, imag-c.imag);
    }
    Complex operator*(const Complex& c) const {
        return Complex(real*c.real - imag*c.imag,
                        real*c.imag + imag*c.real);
    }

    // friend for double + Complex (left op not our class)
    friend Complex operator+(double d, const Complex& c) {
        return Complex(d+c.real, c.imag);
    }

    void display() const { cout<<real<<"+"<<imag<<"i "; }
};

Complex c1(3,4), c2(1,-2);
(c1+c2).display();    // 4+2i
(c1-c2).display();    // 2+6i
(c1*c2).display();    // 11-2i
(5.0+c1).display();  // 8+4i (friend version)

```

Output/Input Operators (<< and >>)

```

class Complex {
    double real, imag;
public:
    Complex(double r=0, double i=0): real(r), imag(i) {}

    // << MUST be friend (left operand is ostream)
    friend ostream& operator<<(ostream& os, const Complex& c) {
        if(c.imag >= 0) os << c.real << "+" << c.imag << "i";
        else           os << c.real << c.imag << "i";
        return os;    // Return ref enables: cout << a << b
    }

    // >> MUST be friend (left operand is istream)
    friend istream& operator>>(istream& is, Complex& c) {
        cout<<"Real: "; is>>c.real;
        cout<<"Imag: "; is>>c.imag;
        return is;    // Return ref enables: cin >> a >> b
    }
};

Complex c1(3,4), c2;
cout << c1 << endl;    // 3+4i
cin >> c2;              // Takes input
cout << c1 << c2 << endl; // Chaining works

```

Prefix and Postfix Increment/Decrement (++/--)


```

class Counter {
    int value;
public:
    Counter(int v=0) : value(v) {}

    // PREFIX ++obj: no dummy parameter, returns reference
    Counter& operator++() {
        ++value;
        return *this; // Return modified object
    }

    // POSTFIX obj++: dummy int parameter, returns copy
    Counter operator++(int) {
        Counter temp = *this; // Save BEFORE incrementing
        ++value;
        return temp;          // Return OLD value
    }

    // PREFIX --
    Counter& operator--() { --value; return *this; }

    // POSTFIX --
    Counter operator--(int) { Counter t=*this; --value; return t; }

    int get() const { return value; }
};

Counter c(5);
Counter c1 = ++c; // c→6, c1=6 (prefix: modify then return)
Counter c2 = c++; // c2=6, c→7 (postfix: return then modify)
cout << c1.get(); // 6
cout << c2.get(); // 6
cout << c.get();  // 7

```

Comparison Operators (==, !=, <, >, <=, >=)

```

class Fraction {
    int num, den;
public:
    Fraction(int n, int d) : num(n), den(d) {}

    bool operator==(const Fraction& f) const {
        return num*f.den == f.num*den;
    }
    bool operator!=(const Fraction& f) const { return !(*this==f); }
    bool operator<(const Fraction& f) const { return num*f.den < f.num*den; }
    bool operator>(const Fraction& f) const { return f < *this; }
    bool operator<=(const Fraction& f) const { return !(f < *this); }
    bool operator>=(const Fraction& f) const { return !(*this < f); }
    void display() { cout<<num<<"/"<<den; }
};

```

```

Fraction a(1,2), b(2,4), c(3,4);
cout << (a==b); // 1 (1/2 == 2/4)
cout << (a<c); // 1 (1/2 < 3/4)
cout << (c>a); // 1

```

Assignment Operator (=)

```

class MyStr {
    char* data; int len;
public:
    MyStr(const char* s="") { len=strlen(s); data=new char[len+1];
    strcpy(data,s); }

    MyStr& operator=(const MyStr& other) {
        if(this == &other) return *this; // Self-assignment guard!
        delete[] data; // Free old memory
        len = other.len;
        data = new char[len+1];
        strcpy(data, other.data);
        return *this; // Enable: a=b=c
    }

    MyStr& operator+=(const MyStr& other) {
        char* t = new char[len+other.len+1];
        strcpy(t,data); strcat(t,other.data);
        delete[] data; data=t; len+=other.len;
        return *this;
    }

    void display() { cout<<data; }
    ~MyStr() { delete[] data; }
};

MyStr s1("Hello"), s2;
s2 = s1; // operator= called
s1 += MyStr(" World"); // operator+= called
s1.display(); // Hello World
s2.display(); // Hello

```

Subscript Operator ([])

```

class SafeArray {
    int* arr; int size;
public:
    SafeArray(int n) : size(n) { arr = new int[n](); }

    // For non-const objects – allows read AND write
    int& operator[](int i) {
        if(i<0 || i>=size) { cout<<"Index "<<i<<" 00B!\n"; exit(1); }
        return arr[i]; // Reference allows: a[i] = 99;
    }
};

```

```

    }

    // For const objects – allows read only
    const int& operator[](int i) const {
        if(i<0 || i>=size) { cout<<"Index 00B!\n"; exit(1); }
        return arr[i];
    }

    ~SafeArray() { delete[] arr; }
};

SafeArray a(5);
a[0]=10; a[1]=20; a[2]=30; // Write: calls non-const operator[]
cout << a[2];             // Read: 30
// a[10] = 5;             // Error: Index 00B

```

Function Call Operator (()) — Functors

```

// A class with operator() becomes a 'functor' (function object)
// Functors can carry state – unlike regular functions

class Multiplier {
    int factor;
public:
    Multiplier(int f) : factor(f) {}

    int operator()(int x) const { return x * factor; }
    int operator()(int x, int y) const { return x * y * factor; }
};

class Distance {
public:
    double operator()(double x1,double y1,double x2,double y2) const {
        double dx=x2-x1, dy=y2-y1;
        return sqrt(dx*dx+dy*dy);
    }
};

int main() {
    Multiplier triple(3);
    cout << triple(5);      // 15 – like calling triple as a function!
    cout << triple(4,2);    // 24

    // Can be used with algorithms
    int arr[]={1,2,3,4,5};
    for(int x:arr) cout << triple(x) << " "; // 3 6 9 12 15

    Distance dist;
    cout << dist(0,0,3,4); // 5.0 (3-4-5 triangle)
}

```

Unary Operator Overloading

```
class Vector2D {
    double x, y;
public:
    Vector2D(double x=0, double y=0): x(x), y(y){}

    // Unary minus: negate
    Vector2D operator-() const { return Vector2D(-x, -y); }

    // Unary plus: identity
    Vector2D operator+() const { return *this; }

    // Logical NOT: true if zero vector
    bool operator!() const { return (x==0 && y==0); }

    // Magnitude (not standard, but shows custom unary)
    double magnitude() const { return sqrt(x*x+y*y); }

    void display() const { cout<<"("<<x<<", "<<y<<" "; }
};

Vector2D v(3,4);
Vector2D neg = -v; neg.display();    // (-3, -4)
Vector2D pos = +v; pos.display();    // (3,4)
cout << !v;           // 0 (not zero vector)
cout << !Vector2D(); // 1 (zero vector)
```

5. Quick Reference Tables

All Operator Overloading Summary

Operator	Example Use	Member/Friend	Key Notes
+ - * /	a + b	Either (Friend preferred)	Return new object by value
== != < > <= >=	a == b	Either	Return bool
++ -- (prefix)	++a	Member	Return *this (reference)
++ -- (postfix)	a++	Member	Dummy int param; return copy
<<	cout << obj	Friend (must)	Return ostream& for chaining
>>	cin >> obj	Friend (must)	Return istream& for chaining
=	a = b	Member (must)	Check self-assign; return *this
[]	a[i]	Member	Return reference (int&)
()	obj(args)	Member	Creates functor; can

			overload
Unary - +	- obj	Member	No extra parameter

Constructor Types Summary

Type	Syntax	When Called
Default	Class() {}	Object created with no args: Class c;
Parameterized	Class(type p) {}	With args: Class c(5);
Copy	Class(const Class& o) {}	Copied: Class b = a; or Class b(a);
Dynamic	Uses new inside body	When heap allocation needed in ctor

OOP Key Concepts at a Glance

Concept	Definition	C++ Mechanism
Encapsulation	Bundling data + methods; restricting access	private data + public methods
Abstraction	Showing only essential details	Access specifiers, abstract classes
Inheritance	Derived class reuses base class properties	class Derived : public Base {}
Polymorphism	One interface, many behaviors	Overloading (compile), virtual (runtime)
Static Member	Class-wide shared variable/function	static keyword
Friend	Non-member access to private data	friend keyword
Dynamic Memory	Runtime heap allocation	new / delete operators